

pytest

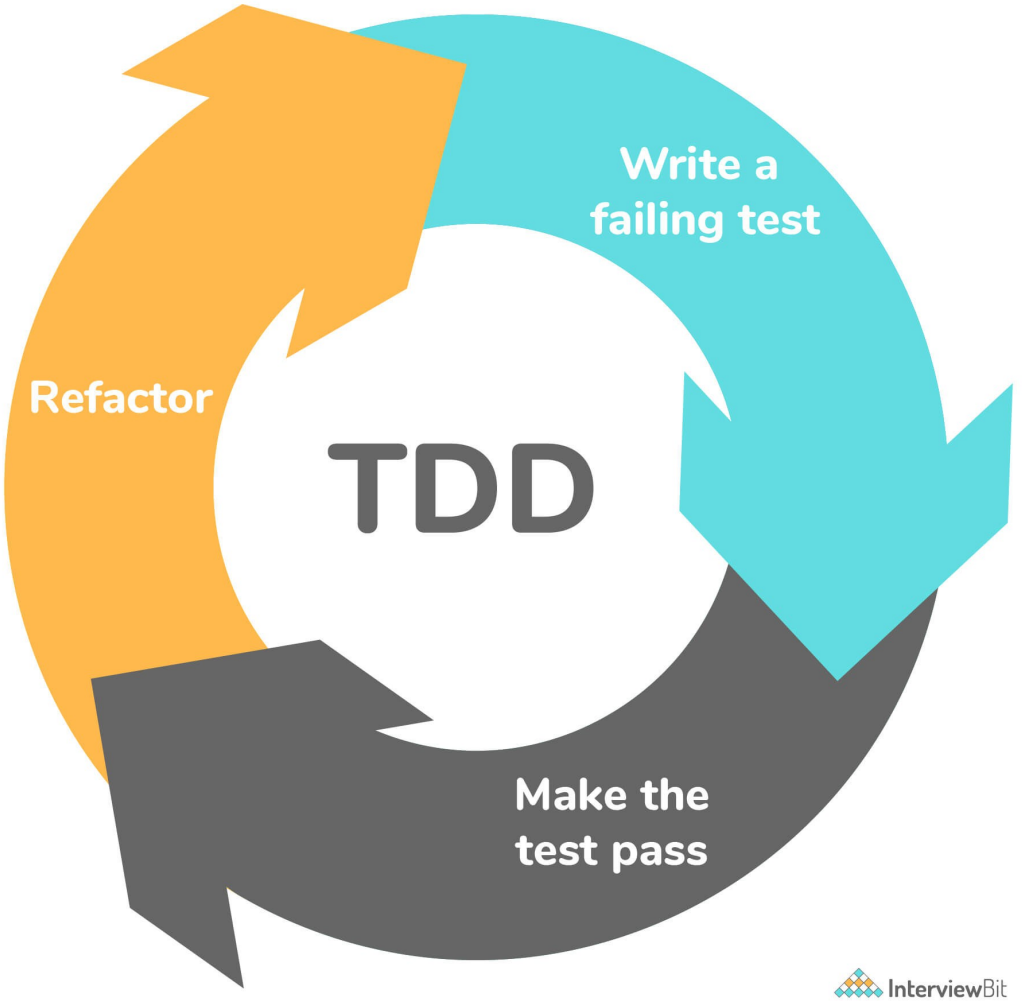
Принцип из **«Extreme Programming»**

«Сначала тест, потом код»

Тот же тест, но на PyTest

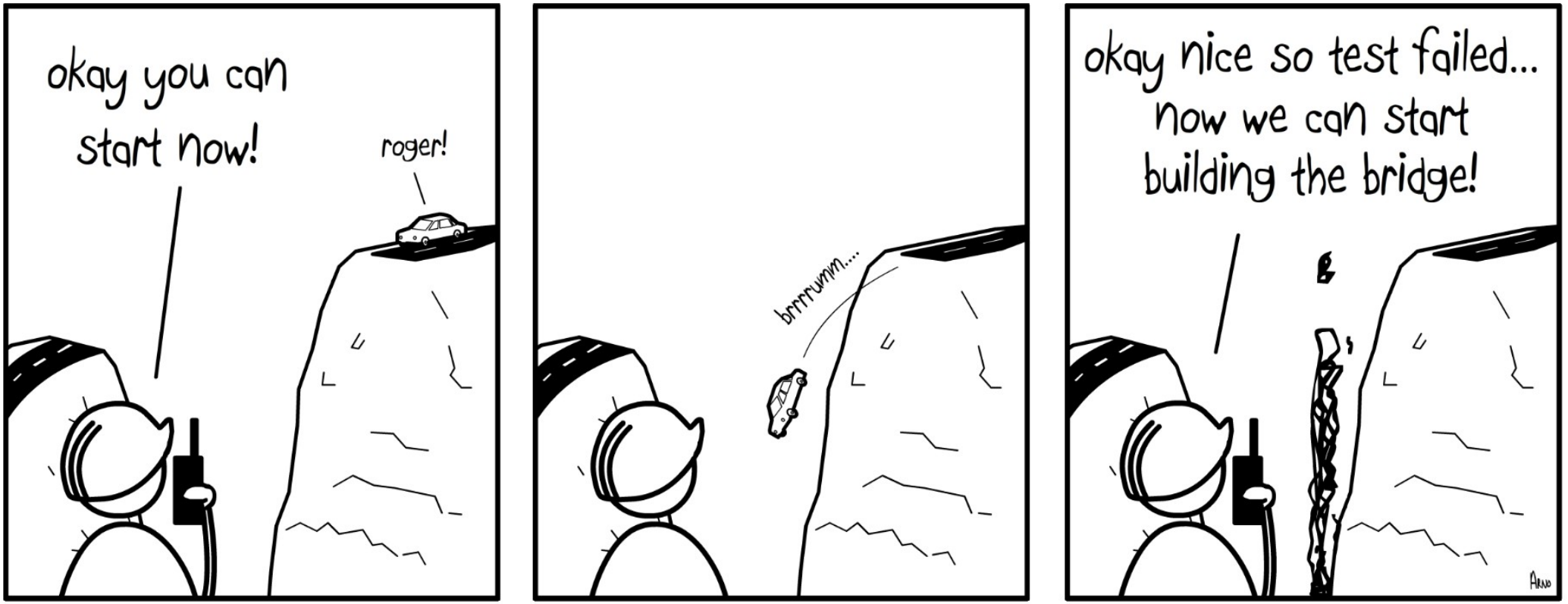
```
def test_add():
    assert add(2, 2) == 4
```

Test-Driven Development как часть **XP**



TDD

Разработка через тестирование — это методология, которая основывается на повторении коротких циклов разработки: изначально пишется тест, который **покрывает** желаемое изменение, затем пишется программный код, который реализует желаемое поведение системы и позволит пройти написанный тест.


okay you can start now!
roger!
brrrumm...
okay nice so test failed... now we can start building the bridge!

Основные виды функционального тестирования покрывает PyTest

Модульное (Unit) — отдельные функции/классы

Интеграционное — взаимодействие модулей, API, СУБД, ф.с.

Регрессионное — не сломалось ли старое после изменений

Дымовое (Smoke) — проверка базовых сценариев

Приёмочное — соответствует ли требованиям заказчика

Системное — вся система целиком

Нефункциональное тестирование

Нагрузочное (Load) — поведение при ожидаемой нагрузке

Стресс-тестирование — поведение за пределами нормы

Тестирование производительности — скорость, отклик

Тестирование безопасности

Тестирование удобства (UX)



https://github.com/yandex/yandex-tank

Классика жанра

Вот так выглядит типичный тест на встроенной библиотеки (**`unittest`**) в Python . Класс, метод setUp, self, assertEqual. Много шума — мало смысла.

```
class TestCalc(unittest.TestCase):
    def test_add(self):
        self.assertEqual(self.calc.add(2, 2), 4)
```

Почему старые методы не работают?

Почему **`unittest`** устарел:

- ✗ Многословный boilerplate
- ✗ Нет нормальной параметризации
- ✗ Фикстуры через setUp/tearDown — боль
- ✗ Отчёт об ошибках — как шифр

Можно проще ?

```
assert calc.add(2, 2) == 4
```

Тот же тест, но на PyTest

```
def test_add():
    assert add(2, 2) == 4
```

Тот же тест, но на PyTest

```
def test_add():
    assert add(2, 2) == 4
```

The reaction to a "BUG"



Ключевые концепции

Тест - функция, начинающаяся с test_, которую находит и запускает pytest

Фикстура - функция-подготовка ресурсов (БД, клиент, данные) с декоратором @pytest.fixture

Параметризация - запуск одного теста с разными входными данными через @pytest.mark.parametrize

Маркер - метка для теста (@pytest.mark.smoke), позволяет выборочно запускать тесты

Assert - обычный Python assert — pytest сам покажет, что и где упало

Conftest – файл conftest.py, где хранят общие фикстуры для всей папки

Тест

файл calc.py

```python
def add(a, b):
    """Складывает два числа."""
    return a + b
```

файл test_calc.py

```python
def test_add():
    assert add(2, 2) == 4
```

Фикстура

```python
import pytest


@pytest.fixture
def number():
    return 42


def test_number(number):
    assert number == 42
```

pytest видит аргумент number в тесте, находит фикстуру с таким именем и подставляет её результат.

Фикстура возвращает словарь

```python
import pytest


@pytest.fixture
def user():
    return {"name": "Alice", "age": 30}


def test_user_name(user):
    assert user["name"] == "Alice"


def test_user_age(user):
    assert user["age"] == 30
```

Одна фикстура используется в нескольких тестах.
Данные создаются заново для каждого теста.

Фикстура возвращает mock db

```python
import pytest
import mongomock


@pytest.fixture
def mongo_db():
    client = mongomock.MongoClient()
    yield client["test_db"]
```

Тест с mock db

```python
def test_insert_and_read(mongo_db):
    users = mongo_db["users"]
    users.insert_one({"name": "Alice", "age": 30})

    result = users.find_one({"name": "Alice"})
    assert result["age"] == 30
    assert users.count_documents({}) == 1
```

Тест с рельной MongoDB

```
@pytest.fixture
def mongo_db():
    if os.getenv("USE_REAL_MONGO"):
        from pymongo import MongoClient
        client = MongoClient("mongodb://localhost:27017/")
    else:
        import mongomock
        client = mongomock.MongoClient()

    db = client["test_db"]
    yield db
    client.close()
```

Зависимость фикстур

```python
import pytest


@pytest.fixture
def base_url():
    return "https://api.example.com"


@pytest.fixture
def endpoint(base_url):
    return f"{base_url}/users"


def test_endpoint(endpoint):
    assert endpoint == "https://api.example.com/users"
```

Где pytest ищет фикстуры ?

1. Прямо в том же файле, что и тест
2. В conftest.py рядом с тестами
3. В conftest.py в корне проекта

```
project/
├── tests/
|   ├── conftest.py       # ← фикстуры для всей папки tests/
|   └── test_calc.py
```

Параметризация

```python
@pytest.mark.parametrize("a,b,expected", [
    (2, 2, 4),
    (0, 0, 0),
    (-1, 1, 0),
])
def test_add(a, b, expected):
    assert add(a, b) == expected
```

Один тест на многих данных

Маркер

```python
import pytest

@pytest.mark.smoke
def test_app_starts():
    assert app.is_alive() is True

@pytest.mark.smoke
def test_db_connection():
    assert db.ping() is True

@pytest.mark.smoke
def test_login_endpoint():
    response = client.post("/login", json={"user": "admin", "pass": "123"})
    assert response.status_code == 200
```

Система вообще работает?
pytest -m smoke

conftest.py

```python
import pytest
import requests


@pytest.fixture(scope="session")
def http_session():
    """Одна сессия requests на весь прогон (переиспользует TCP-соединения)."""
    session = requests.Session()
    session.headers.update({"User-Agent": "pytest-tests/1.0"})
    yield session
    session.close()


@pytest.fixture
def api_base_url():
    """Базовый URL тестового API."""
    return "https://jsonplaceholder.typicode.com"
```

PyTest уже сегодня в твоем проекте

- pip install pytest
- Написать постой тест на функцию
- Запустить pytest -v
- Создать .yaml файл workflow
- Запустить тесты в GitHub Actions